I102 - Paradigmas de Programación

OOP – PATRONES DE DISEÑO

8

Prof. Mariano Scaramal

Ingeniería en Inteligencia Artificial

Que conceptos de C++?

- Patrones de Diseño Creacionales
  - Singleton
  - Factory Method
  - Builder
- Patrones de Diseño Estructurales
  - Adapter
  - Composite
  - Decorator
- Patrones de Diseño De Comportamiento
  - Observer
  - Strategy
  - Command

¿Qué son Patrones de Diseño?

Son soluciones ya probadas y reutilizables a problemas frecuentes en el desarrollo del software.

No hay que tomarlo como que es código ya preparado, sino como guías de diseño que orientan la implementación.

Dada su difusión, estas ayudan a la mantenibilidad, escalabilidad y claridad del código.

También, ayudan a la consistencia del código y a la comunicación de los desarrolladores dado que los patrones son conocidos entre ellos.

Algunos desarrolladores lo ven como una receta ya verificada por muchos desarrolladores que han resuelto un problema dado varias veces.

Están pensados para evitar errores comunes de diseño y permitir adaptación a cambios futuros.

Clasificación de Patrones

Universidad de SanAndrés

En la bibliografía recomendada, "*Design Patterns: Elements of Reusable Objects-Oriented Software*" de Gamma, Helm, Johnson y Vlissides (Gang of Four), hay definidos 23 patrones de diseño que se agruparon en tres familias:

1. Creacionales: Encapsulan la creación de objetos, desacoplando el código cliente de las clases concretas. Por ejemplo: Singleton, Factory, etc.
2. Estructurales: Definen como se componen clases y objetos para formar estructuras más complejas, manteniendo flexibilidad. Por ejemplo: Adapter, Composite, etc.
3. De Comportamiento: Definen cómo los objetos interactúan y distribuyen responsabilidades. Por ejemplo: Observer, Strategy, etc.

Si bien el diseño de patrones podría estar incluida dentro de varias materias de diseño de software y arquitectura, en el curso veremos algunos de estos patrones y cómo pueden combinarse para resolver problemas de diseño más complejos.

Patrones de Diseño Creacionales: Singleton

El objetivo de este patrón de diseño es garantizar que una clase tenga una única instancia y proporcionar un punto global de acceso a ella cuando esta es necesaria.

Se suele utilizar cuando se necesita un único objeto para controlar algún recurso compartido o se desea tener una única fuente de verdad para una configuración (ejemplo, el menú de video para una programa).

Estructura de la clase implica tener un constructor privado, un método estático *getInstance()* que devuelve la instancia única de la clase.

Es importante que el objeto no se duplique, para ello debe evitarse:

1. Copia por creación o por copy constructor.
2. Copia por asignación, mediante el operador de asignación.
3. Copia por medio de la herencia.

Singleton - Ejemplo

```
#include <iostream>
using namespace std;

class Singleton final {   // No se puede heredar
private:
   // Constructor privado
   Singleton() { cout << "Constructor Singleton\n"; }
   ~Singleton() {}
   // Prohibe el copy constructor
   Singleton(const Singleton&) = delete;
   // Prohibe asignación
   Singleton& operator=(const Singleton&) = delete;

public:
   // Desde C++11, así es como la instancia se crea thread-safe
   static Singleton& getInstance() {
      static Singleton instance;   // static local: thread-safe
      return instance;
   }

   void showMessage() { cout << "Soy Singleton!" << endl;}
};
```

```
int main() {
   Singleton& s1 = Singleton::getInstance();
   Singleton& s2 = Singleton::getInstance();

   s1.showMessage();
   cout << (&s1 == &s2 ? "Misma instancia" : "Distinta instancia")
         << endl;
   // Lo siguiente NO compila, porque está prohibido:
   // Singleton s3 = s1;      // Copia por creación no permitida
   // Singleton s4;           // El constructor es privado
   // s4 = s1;                // Copia por asignación no permitida
   return 0;
}
```

Imprime:

Constructor Singleton
Soy Singleton!
Misma instancia

Patrones de Diseño Creacionales: Factory

El objetivo de este patrón de diseño es encapsular la creación de objetos, permitiendo decidir en tiempo de ejecución qué tipo de objeto instanciar.

Se utiliza cuando una clase no puede anticipar la clase exacta del objeto a crear o cuando se quiere desacoplar la creación de objetos de su uso.

Se lo puede encontrar en las GUI de aplicaciones donde se crean distintos tipos de botones según el sistemas operativos, asignando la factory adecuada.

También se utiliza cuando el tipo de objeto a crear depende de condiciones en tiempo de ejecución.

Se implementa mediante una clase abstracta o una interfaz que define un método de creación, cuya lógica puede variar según la implementación.

El término Factory se utiliza de forma general para referir a un patrón que encapsula la creación de objetos. Existen variantes más específicas como Factory Method y Abstract Factory.

Factory - Ejemplo

```
#include <iostream>
#include <vector>
using namespace std;

class Car {
public:
   virtual void drive() = 0;
   virtual ~Car() {}
};

class ElectricSedan : public Car {
public:
   void drive() override { cout << "Manejando un Sedan
                                                eléctrico\n";}
};

class ElectricSUV : public Car {
public:
   void drive() override { cout << "Manejando un SUV
                                                eléctrico\n";}
};
```

```
class GasSedan : public Car {
public:
   void drive() override {
            cout << "Manejando un Sedan a
                                          gasolina\n";}
};

class GasSUV : public Car {
public:
   void drive() override {
      cout << "Conduciendo un SUV a gasolina\n";
   }
};

class CarFactory { // Clase abstracta o interfaz
public:     // 0=Sedan, 1=SUV
   virtual Car* createCar(int type) = 0;
   virtual ~CarFactory() {}
};
```

Factory - Ejemplo

```
class ElectricCarFactory : public CarFactory {
public:
   Car* createCar(int type) override {
      // Si type es cero, se crea un Sedan
      if (type == 0) return new ElectricSedan();
      // Si type es uno, se crea un SUV
      else return new ElectricSUV();
   }
};

class GasCarFactory : public CarFactory {
public:
   Car* createCar(int type) override {
      // Si type es cero, se crea un Sedan
      if(type == 0) return new GasSedan();
      // Si type es uno, se crea un SUV
      else return new GasSUV();
   }
};
```

```
int main() {
   vector<CarFactory*> factories = {
                                    new ElectricCarFactory(),
                                    new GasCarFactory()
                                   };

   vector<int> types = {0, 1};  //Sedan=0, SUV=1

   vector<Car*> cars; // Guardar punteros para luego liberarlos

   for(auto factory : factories)
      for(auto type : types) {
         Car* car = factory->createCar(type);
         car->drive();
         cars.push_back(car);
      }

   // Liberar memoria
   for(auto car : cars) delete car;
   for(auto factory : factories) delete factory;
}
```

Patrones de Diseño Creacionales: Builder

El objetivo de este patrón de diseño es separar el proceso de construcción de un objeto complejo de su representación final, permitiendo crear distintos tipos y configuraciones del objeto paso a paso utilizando el mismo proceso de construcción.

Se suele utilizar cuando un objeto requiere múltiples pasos de creación o cuando existen muchas combinaciones posibles de sus atributos, evitando constructores extensos o difíciles de mantener.

Se lo puede encontrar, por ejemplo, en la construcción de documentos, reportes o configuraciones complejas, donde el mismo proceso puede generar representaciones distintas según los parámetros utilizados.

También se utiliza cuando es necesario garantizar un orden específico en la construcción o validar el estado del objeto antes de su uso.

Se implementa mediante una interfaz o clase abstracta que define los pasos de construcción, junto con una o más clases concretas que implementan dichos pasos, y opcionalmente una clase *Director* que coordina el proceso de construcción.

Builder - Implementación General

```
#include <iostream>
#include <memory>

// Product
class Product {
   /* Es el objeto complejo cuya construcción se quiere
   desacoplar de su representación. */
};

/* Interfaz Builder:
 - Define los pasos del proceso de construccion
 - Oculta cómo se construye el producto, solo se encarga
   de definir esto y su implementación de como hacerlo.
 - Presenta un método para obtener el producto.
*/
class Builder {
public:
   virtual ~Builder() {}
   virtual void buildPartA() = 0;
   virtual void buildPartB() = 0;
   virtual std::unique_ptr<Product> getProduct() = 0;
};
```

```
/* Implementacion de Builder:
 - Implementa cómo se construyen las distintas partes
 - Transfiere el ownership utilizando un unique_ptr
*/
class ConcreteBuilder : public Builder {
   std::unique_ptr<Product> product;
public:
   ConcreteBuilder() {
      product = std::make_unique<Product>();
   }

   void buildPartA() override { /* construir parte A */ }
   void buildPartB() override { /* construir parte B */ }

   std::unique_ptr<Product> getProduct() override {
      auto result = std::move(product);
      product = std::make_unique<Product>(); // reset
      return std::move(product);
   }
};
```

Builder - Implementación General (cont'd)

```
/* Director:
 - Define la lógica del proceso de construcción.
 - No conoce detalles del producto ni del builder.
 - Permite utilizar el mismo proceso con distintos builders.

   Ejemplo: Los autos de baja y alta gama tienen el mismo
   proceso de construccion, pero se utilizan distintos
   materiales para construirlos.
*/
class Director {
   Builder* builder;
public:
   void setBuilder(Builder* b) { builder = b; }

   std::unique_ptr<Product> construct() {
      builder->buildPartA();
      builder->buildPartB();
      return builder->getProduct();
   }
};
```

```
int main() {
   ConcreteBuilder builder;
   Director director;
   director.setBuilder(&builder);

   std::unique_ptr<Product> product = director.construct();
   std::cout << "Producto construido usando Builder.\n";

   return 0;
}
```

Builder - Ejemplo

```
#include <iostream>
#include <string>
#include <memory>
using namespace std;

class Robot {
   std::string sensor, actuator;
   std::string battery, navigation;
public:
   void setSensor(const string& s) { sensor = s; }
   void setActuator(const string& a) { actuator = a; }
   void setBattery(const string& b) { battery = b; }
   void setNavigation(const string& n) { navigation = n; }

   void showSpecs() {
      std::cout << "Robot specs:\n";
      std::cout << "- Sensor: " << sensor << "\n";
      std::cout << "- Actuator: " << actuator << "\n";
      std::cout << "- Battery: " << battery << "\n";
      std::cout << "- Navigation: " << navigation << "\n";
   }
};
```

```
class RobotBuilder {
protected:
   unique_ptr<Robot> robot;
public:
   RobotBuilder() { robot = make_unique<Robot>(); }
   virtual ~RobotBuilder() {}
   virtual void buildSensor()       = 0;
   virtual void buildActuator()     = 0;
   virtual void buildBattery()      = 0;
   virtual void buildNavigation() = 0;
   unique_ptr<Robot> getRobot() { return move(robot); }
};

class ExplorerRobotBuilder : public RobotBuilder {
public:
   void buildSensor()     override { robot->setSensor("Lidar"); }
   void buildActuator()   override { robot->setActuator("Motor"); }
   void buildBattery()    override { robot->setBattery("Li-ion"); }
   void buildNavigation() override {
                                   robot->setNavigation("RandomWalk");
   }
};
```

Builder – Ejemplo (cont'd)

```
class GPSRobotBuilder : public RobotBuilder {
public:
   void buildSensor()     override { robot->setSensor("Camera"); }
   void buildActuator()   override { robot->setActuator("Servo"); }
   void buildBattery()    override { robot->setBattery("NiMH"); }
   void buildNavigation() override { robot->setNavigation("GPS"); }
};

class Director {
   RobotBuilder* builder;
public:
   void setBuilder(RobotBuilder* b) { builder = b; }
   unique_ptr<Robot> construct() {
      builder->buildSensor();
      builder->buildActuator();
      builder->buildBattery();
      builder->buildNavigation();
      return builder->getRobot();
   }
};
```

```
int main() {
   Director director;

   ExplorerRobotBuilder explorerBuilder;
   director.setBuilder(&explorerBuilder);
   auto explorerRobot = director.construct();
   explorerRobot->showSpecs();

   GPSRobotBuilder gpsBuilder;
   director.setBuilder(&gpsBuilder);
   auto gpsRobot = director.construct();
   gpsRobot->showSpecs();

   return 0;
}
```

Resumen Patrones de Diseño Creacionales

Como se ha visto, los patrones de diseño creacionales tiene como objetivo el separar la creación de objetos de su uso, desacoplando el código cliente de las clases concretas.

Alta Cohesión:

- La lógica de creación se concentra en clases específicas (factories y builders).
- Las clases cliente se enfocan exclusivamente en su responsabilidad funcional.

Bajo Acoplamiento:

- El cliente depende de interfaces o abstracciones, no de clases concretas.
- Se reduce la dependencia directa a constructores y tipos específicos.

Evitan Código Duplicado:

- La creación compleja o repetitiva se implementa una sola vez.
- Se reducen la duplicación de lógica de creación e inicialización.

Bajo Impacto Ante Cambios:

- Cambios en la forma de crear objetos no afectan al código cliente que los utiliza.
- Es posible agregar nuevas variantes sin modificar código existente.

Patrones de Diseño Estructurales: Adapter

El objetivo de este patrón de diseño es permitir que interfaces incompatibles trabajen juntas sin modificar las clases existentes. También se lo conoce como wrapper.

Se utiliza cuando se quieren reutilizar clases existentes pero su interfaz no coincide con la del nuevo código o aquel con el que desea interactuar.

Prácticamente se la utiliza para integración de sistemas legacy o librerías a terceros, etc.

El cliente interactúa con una interfaz esperada (*Target*). El Adapter traduce las llamadas hacia la interfaz existente (*Adaptee*). La idea es que el adapter implemente "*target*" y traduzca la llamada para colaborar con "*adaptee*".

El utilizar este patrón puede aumentar la complejidad si se utilizan demasiados adapters. No obstante, también facilita la reutilización de código.

Adapter – Implementación General

```
#include <iostream>
using namespace std;

// Interfaz esperada
class Target {
public:
    virtual void request() = 0;
};

// Clase existente incompatible
class Adaptee {
public:
    void specificRequest() {
        cout << "Llamada específica de Adaptee\n";
    }
};
```

```
// Adapter convierte la interfaz
class Adapter : public Target {
private:
    // Agrega la clase legacy como atributo
    Adaptee* adaptee;
public:
    Adapter(Adaptee* a) : adaptee(a) {}
    // Implementa la nueva interfaz con la legacy
    void request() override {
        adaptee->specificRequest();
    }
};

int main() {
    Adaptee* adaptee = new Adaptee();
    Target* target = new Adapter(adaptee);
    target->request();
    delete adaptee;  // Son raw pointers
    delete target;   // Son raw pointers
}
```

Adapter – Ejemplo de Aplicación

```
#include <iostream>
#include <string>
using namespace std;

// Interfaz o clase abstracta moderna
class MessageSender {
public:
   virtual void sendMessage(const string& recipient,
                            const string& content) = 0;
   virtual ~MessageSender() {}
};
```

```
// Librería antigua o legacy que no queremos modificar,
class LegacyEmailSender {//(Adaptee)
public:
   void sendEmail(const string& emailAddress,
                  const string& message) {
      cout << "Enviando EMAIL a " << emailAddress
           << ": " << message << endl;
   }
};
```

El cliente trabaja con nombres de usuario, pero la librería legacy solo sabe enviar emails a direcciones completas.

El adapter transforma el nombre de usuario en una dirección de email válida y delega el envío

Adapter – Ejemplo de Aplicación (cont'd)

```
// Adapter
class EmailAdapter : public MessageSender {
private:
   LegacyEmailSender* legacy;
public:
   EmailAdapter(LegacyEmailSender* legacySender)
                    : legacy(legacySender) {}

   void sendMessage(const string& recipient,
                          const string& content)
                         override {

      string newEmail = nameUsr + "@aol.com";
      legacy->sendEmail(newEmail, content);

   }
};
```

```
int main() {
   LegacyEmailSender* oldEmailLib = new LegacyEmailSender();
   MessageSender* sender = new EmailAdapter(oldEmailLib);

   sender->sendMessage("juan",
       "Hola Juan, este es un mensaje moderno adaptado!");

   delete sender;
   delete oldEmailLib;
   return 0;
}
```

Imprime:

Enviando EMAIL a juan@aol.com: Hola Juan,
este es un mensaje moderno adaptado!

Patrones de Diseño Estructurales: Composite

El objetivo de este patrón de diseño es componer objetos en estructuras jerárquicas de árbol y permitir que clientes traten objetos individuales y compuestos de manera uniforme. También se lo conoce como “Part-Whole”.

Se lo utiliza cuando se desea representar relaciones parte-todo (archivos-carpetas, opciones-menú, etc). También cuando no se quiere que el cliente tenga que distinguir entre objetos “simples” y “compuestos”.

Los participantes en este patrón son:

- *Componente*: que es la interfaz común para hojas y compuestos.
- *Hojas*: implementa objetos “atómicos”, es decir, sin hijos.
- *Compuesto*: contiene y gestiona hijos que también son “*Componentes*”
- *Cliente*: utiliza “*Componentes*” sin importar que sean hojas o compuestos.

La idea es que un cliente invoca operaciones sobre un componente, si este es una hoja, ejecuta directamente la operación, sino la delega a sus hijos.

Composite – Impl. General como Sistema de Archivos

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Componente
class FileSystemComponent {
public:
   virtual void show(int indent = 0) = 0;
   virtual ~FileSystemComponent() {}
};

// Hoja
class File : public FileSystemComponent {
   string name;
public:
   File(const string& n) : name(n) {}
   void show(int indent = 0) override {
      cout << string(indent, '-') << "File: " << name
            << endl;
   }
};
```

```
// Composite
class Directory : public FileSystemComponent {
   string name;
   vector<FileSystemComponent*> children;
public:
   Directory(const string& n) : name(n) {}

   void add(FileSystemComponent* c) {
      children.push_back(c);
   }

   void show(int indent = 0) override {
      cout << string(indent, '-') << "Dir: " << name << endl;

      for (auto c : children)
         c->show(indent + 2);
   }
};
```

Composite – Sistema de Archivos (cont'd)

```
// Cliente
int main() {
    Directory root("root");
    root.add(new File("readme.txt"));

    // Puntero porque se agregara a "root"
    Directory* src = new Directory("src");
    src->add( new File("main.cpp") );
    src->add( new File("utils.cpp") );

    root.add(src);
    // Muestra la estructura en forma de árbol
    root.show();

    return 0;
}
```

Imprime:

```
Dir: root
--File: readme.txt
--Dir: src
----File: main.cpp
----File: utils.cpp
```

Composite – Lenguaje de Comandos

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// Componente base
class Command {
public:
   virtual void execute() = 0;
   virtual ~Command() = default;
};

// Hoja: comando simple
class PrintCommand : public Command {
   std::string message;
public:
   PrintCommand(const std::string& msg) : message(msg) {}

   void execute() override {
      std::cout << "PRINT: " << message << std::endl;
   }
};
```

```
// Hoja: otro comando
class AddCommand : public Command {
   int a, b;
public:
   AddCommand(int x, int y) : a(x), b(y) {}
   void execute() override {
      std::cout << "ADD: " << a << " + " << b << " = "
                << (a+b) << std::endl;
   }
};

// Composite: un bloque de comandos
class BlockCommand : public Command {
   std::vector<std::unique_ptr<Command>> commands;
public:
   void add(std::unique_ptr<Command> cmd) {
      commands.push_back(std::move(cmd));
   }
   void execute() override {
      for (auto& cmd : commands)
         cmd->execute();
   }
};
```

Composite – Lenguaje de Comandos (cont'd)

```
// Otro Composite: Loop de comandos
class LoopCommand : public Command {
   int times;
   std::unique_ptr<Command> body;
public:
   LoopCommand(int n,
                        std::unique_ptr<Command> b) :
                        times(n),
                        body(std::move(b)) {}

   void execute() override {
      for (int i = 0; i < times; i++)
         body->execute();
   }
};
```

```
int main() {
   // Construcción del programa
   auto program = std::make_unique<BlockCommand>();

   program->add(std::make_unique<PrintCommand>("Inicio del
                                                programa"));
   program->add(std::make_unique<AddCommand>(3, 5));

   // Un loop que repite un bloque
   auto loopBlock = std::make_unique<BlockCommand>();
   loopBlock->add(std::make_unique<PrintCommand>("Dentro del loop"));
   loopBlock->add(std::make_unique<AddCommand>(10, 20));

   program->add(std::make_unique<LoopCommand>(3,
                                              std::move(loopBlock)));
   program->add(std::make_unique<PrintCommand>("Fin del programa"));

   // Ejecutar todo
   program->execute();

   return 0;
}
```

Patrones de Diseño Estructurales: Decorator

El objetivo de este patrón es agregar responsabilidades o comportamientos a un objeto de forma dinámica, sin modificar su clase original.

Se suele utilizar cuando se desea extender la funcionalidad de objetos individuales en tiempo de ejecución, evitando la creación de una gran jerarquía de clases derivadas.

Se lo puede encontrar, por ejemplo, en sistemas de interfaces gráficas, donde se agregan funcionalidades como bordes, scroll o estilos visuales a componentes básicos sin alterar su implementación original.

También es común en sistemas donde se requiere combinar y encadenar múltiples comportamientos de forma flexible.

Se implementa mediante una interfaz o clase base común, junto con una o más clases decoradoras que envuelven al objeto original y delegan las operaciones, agregando comportamiento antes o después de la llamada.

Decorator – Implementación General

```
#include <iostream>
#include <memory>

/* Component: Esta es la interfaz que deben usar el
componente y el decorator. */
class Component {
public:
   virtual ~Component() {}
   virtual void operation() = 0;
};

/* ConcreteComponent: objeto original que implementa el
comportamiento a decorar. No sabe que puede ser
decorado. */
class ConcreteComponent : public Component {
public:
   void operation() override { // Clase no abstracta
      std::cout << "Operación básica del componente.\n";
   }
};
```

```
/* Decorator:
 - Hereda de Component
 - Posee un puntero a Component
 - Puede delegar la operación al objeto envuelto
 - Los decoradores concretos agregan comportamiento
adiccional.*/
class Decorator : public Component {
protected:
   std::unique_ptr<Component> component;
public:
   Decorator(std::unique_ptr<Component> c) :
                                    component(std::move(c)) {}
}; // Clase abstracta, no imploementa operation()
```

Decorator – Implementación General (cont'd)

```
/* ConcreteDecorator:
 - Redefine operation(), agrega funcionalidad antes y
   después.
 - El comportamiento se extiende sin modificar la clase.
 - Los decoradores pueden agregarse en cualquier
   orden, combinarse o encadenarse.
*/
class ConcreteDecorator : public Decorator {
public:
    ConcreteDecorator(std::unique_ptr<Component> c) :
                                    Decorator(std::move(c)) {}

    void operation() override {
        std::cout << "Antes de la operación.\n";
        component->operation(); // Componente envuelto
        std::cout << "Después de la operación.\n";
    }
};
```

```
int main() {
    // Componente base
    std::unique_ptr<Component> base =
                    std::make_unique<ConcreteComponent>();

    // Decorador envuelve al componente
    std::unique_ptr<Component> decorated =
     std::make_unique<ConcreteDecorator>(std::move(base));

    // Ejecutar
    decorated->operation();
    return 0;
}
```

Decorator – Ejemplo

```
#include <iostream>
#include <memory>
#include <string>

// Component
class Mensaje {
public:
   virtual ~Mensaje() {}
   virtual void enviar(const std::string& texto) = 0;
};

// ConcreteComponent
class MensajeSimple : public Mensaje {
public:
   void enviar(const std::string& texto) override {
      std::cout << "Enviando: " << texto << std::endl;
   }
};
```

```
// Decorator
class MensajeDecorator : public Mensaje {
protected:
   std::unique_ptr<Mensaje> mensaje;
public:
   MensajeDecorator(std::unique_ptr<Mensaje> m) :
      mensaje(std::move(m)) {}
};

// ConcreteDecorator 1
class MensajeEncriptado : public MensajeDecorator {
public:
   MensajeEncriptado(std::unique_ptr<Mensaje> m) :
      MensajeDecorator(std::move(m)) {}

   void enviar(const std::string& texto) override {
      std::string encriptado = "[Encriptado] " + texto;
      mensaje->enviar(encriptado);
   }
};
```

Decorator – Ejemplo (cont'd)

```
// ConcreteDecorator 2
class MensajeConLogging : public MensajeDecorator {
public:
   MensajeConLogging(std::unique_ptr<Mensaje> m) : MensajeDecorator(std::move(m)) {}

   void enviar(const std::string& texto) override {
      std::cout << "[LOG] Preparando mensaje...\n";
      mensaje->enviar(texto);
   }
};

int main() {
   std::unique_ptr<Mensaje> mensaje = std::make_unique<MensajeSimple>();

   // Decoramos con logging y encriptación
   mensaje = std::make_unique<MensajeConLogging>(std::make_unique<MensajeEncriptado>(std::move(mensaje)));
   mensaje->enviar("Hola, mundo!");

   return 0;
}
```

Resumen Patrones de Diseño Estructurales

Los patrones de diseño estructurales tienen como objetivo organizar y componer objetos, definiendo relaciones entre ellos para formar estructuras flexibles y extendibles:

Alta Cohesión:

- Cada clase tiene una responsabilidad clara (adaptar, componer, extender).
- Las responsabilidades estructurales no se mezclan con la lógica de negocio.

Bajo Acoplamiento:

- El cliente interactúa con interfaces estables, no con estructuras internas.
- Se oculta la complejidad de composición y compatibilidad.

Evitan Código Duplicado:

- Las soluciones de adaptación y composición se encapsulan en un único lugar.
- Se evita repetir lógica de compatibilidad o combinación de comportamientos.

Bajo Impacto Ante Cambios:

- Cambios en la estructura interna no afectan a los clientes.
- Es posible extender funcionalidades sin modificar clases existentes.

Patrones de Diseño de Comportamiento: Observer

El objetivo de este patrón de diseño es definir una dependencia de uno a muchos entre objetos, de modo que cuando el sujeto cambia su estado, todos sus dependientes son notificados automáticamente. Este patrón también se lo asocia con el modelo publish-subscribe.

Se utiliza cuando un cambio en un sujeto debe reflejarse en otros objetos sin que exista un acoplamiento fuerte entre ellos.

Un ejemplo es una aplicación de clima que actualiza múltiples displays cuando cambian los datos del sensor. O sistemas de eventos, donde una acción dispara múltiples respuestas.

La estructura contiene un *Sujeto* que mantiene una lista de O*bservadores*, una interfaz O*bservador* que define el método *update()*, y observadores concretos que reaccionan a los cambios del sujeto.

Observer – Implementación General

```
#include <iostream>
#include <vector>
using namespace std;

class Observer {
public:
   virtual void update(int value) = 0;
};

class Subject { // Quien genera los update
   vector<Observer*> observers;
   int state;
public:
   void attach(Observer* o) { observers.push_back(o); }

   void setState(int s) {
      state = s;
      for (auto o : observers) o->update(state);
   }
};
```

```
// Interesados en conocer los updates
class ConcreteObserver : public Observer {
   string name;
public:
   ConcreteObserver(string n) : name(n) {}

   void update(int value) override {
      cout << name << " recibió actualización: "
            << value << endl;
   }
};

int main() {
   Subject subject;
   ConcreteObserver o1("Obs1"), o2("Obs2");
   subject.attach(&o1);
   subject.attach(&o2);

   subject.setState(10);
   subject.setState(20);
}
```

Observer – Simulador de Eventos del Mouse

```
#include <iostream>
#include <vector>
#include <string>

// Interfaz del Observer
class EventListener {
public:
   virtual void onEvent(const std::string& message) = 0;
};

// Sujeto que genera eventos
class EventSource {
   std::vector<EventListener*> listeners;
public:
   void addListener(EventListener* listener) {
      listeners.push_back(listener);
   }
   void triggerEvent(const std::string& message) {
      for (auto listener : listeners)
         listener->onEvent(message);
   }
};
```

```
// Observador concreto
class Logger : public EventListener {
public:
   void onEvent(const std::string& message) override {
      std::cout << "Logger received: " << message << std::endl;
   } // Se imprime lo que se genera en triggerEvent
};

int main() {
   EventSource source;
   Logger logger;

   source.addListener(&logger);

   source.triggerEvent("User clicked a button");
   source.triggerEvent("User closed the window");

   return 0;
}
```

Patrones de Diseño de Comportamiento: Strategy

El objetivo de este patrón de diseño es definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables sin modificar el cliente que los utiliza. También conocido como Policy.

Se lo utiliza existen varias variantes de un algoritmo o cuando se quiere evitar el uso de condicionales complejos para seleccionar distintos comportamientos.

Por ejemplo, una clase Sorter puede utilizar distintos algoritmos de ordenamiento según el problema o el tipo de dato: QuickSort, MergeSort, etc.

En pocas palabras, permite cambiar el algoritmo en tiempo de ejecución y evita duplicación de código y lógica condicional.

La estructura contiene un objeto "*Context*", que utiliza una estrategia, una interfaz llamada "*Strategy*" común para todos los algoritmos, y clases derivadas de "*Strategy*" que implementa las distintas estrategias.

Strategy – Implementación General

```
#include <iostream>
using namespace std;

class Strategy { // Interfaz Strategy
public:
   virtual void execute() = 0;
};

class StrategyA : public Strategy {// Primer estrategia
public:
   void execute() override {
      cout << "Ejecuto Estrategia A\n";
   }
};

class StrategyB : public Strategy {// Segunda estrategia
public:
   void execute() override {
      cout << "Ejecuto Estrategia B\n";
   }
};
```

```
class Context {
   Strategy* strategy;
public:
   Context(Strategy* s) : strategy(s) {}
   void setStrategy(Strategy* s) { strategy = s; }
   void doWork() { strategy->execute(); }
};

int main() {
   StrategyA a; StrategyB b;
   Context context(&a); // Se pasa estrategia A
   context.doWork();     // Ejecuta estrategia A

   context.setStrategy(&b); // Se pasa estrategia B
   context.doWork(); // Ejecuta estrategia B
}
```

Strategy – Ejemplo de Aplicación

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>

class DiscountStrategy { // Interfaz de Strategy
public:
   virtual double apply(double price) const = 0;
   virtual std::string name() const = 0;
   virtual ~DiscountStrategy() = default;
};

class NoDiscount : public DiscountStrategy {
public:
   double apply(double price) const override {
       return price;   // Sin descuento
   }
   std::string name() const override {
      return "Sin descuento";
   }
};
```

```
class SeasonalDiscount : public DiscountStrategy {
public:
   double apply(double price) const override {
       return price * 0.8; // 20% descuento
   }

   std::string name() const override {
      return "Descuento de temporada";
   }
};

class VIPDiscount : public DiscountStrategy {
public:
   double apply(double price) const override {
       return price * 0.5; // 50% descuento
    }
   std::string name() const override {
      return "Descuento VIP";
   }
};
```

Strategy – Ejemplo de Aplicación (cont'd)

```
// Context
class ShoppingCart {
   std::unique_ptr<DiscountStrategy> strategy;
public:
   void setStrategy(std::unique_ptr<DiscountStrategy> s)
   {
      strategy = std::move(s);
   }

   double checkout(double total) const {
      if ( !strategy ) return total;
      std::cout << "Aplicando: "
                << strategy->name() << "\n";
      return strategy->apply(total);
   }
};
```

```
int main() {
   ShoppingCart cart;

   cart.setStrategy(std::make_unique<NoDiscount>());
   std::cout << "Total: " << cart.checkout(100.0) << "\n";

   cart.setStrategy(std::make_unique<SeasonalDiscount()>);
   std::cout << "Total: " << cart.checkout(100.0) << "\n";

   cart.setStrategy(std::make_unique<VIPDiscount()>);
   std::cout << "Total: " << cart.checkout(100.0) << "\n";

   return 0;
}
```

Imprime:

Total: Aplicando: Sin descuento 100

Total: Aplicando: Descuento de temporada 80

Total: Aplicando: Descuento VIP 50

Patrones de Diseño de Comportamiento: Command

El objetivo de este patrón de diseño es encapsular una solicitud o acción dentro de un objeto independiente llamado comando, permitiendo invocar operaciones sin conocer cómo se implementan ni quién las ejecuta.

Se utiliza cuando se quiere desacoplar el objeto que solicita una operación del objeto que realiza el trabajo real, permitiendo definir, intercambiar o extender acciones sin modificar al invocador.

Se lo puede encontrar, por ejemplo, en sistemas de interfaces gráficas, donde botones, menús o atajos de teclado ejecutan comandos distintos sin que el sistema que los invoca conozca su lógica interna.

También es común en sistemas que requieren organizar, almacenar o ejecutar acciones en un orden determinado, como colas de comandos, macros o historial de operaciones.

Se implementa mediante una interfaz común para los comandos, clases concretas que encapsulan una acción y su receptor, y un objeto invocador que dispara la ejecución del comando a través de dicha interfaz.

Command – Implementación General

```
#include <iostream>
#include <memory>
#include <vector>

/* Interfaz Command:
 - Define una operación abstracta a ejecutar.
 - O bien, representa una solicitud encapsulada como
objeto.
 - El cliente y el invocador solo conocen esta interfaz,
   no las acciones concretas. */
class Command {
public:
    virtual ~Command() {}
    virtual void execute() = 0;
};
```

```
/* Receiver:
 - Es el objeto que sabe como realizar la acción.
 - Contiene la "lógica de negocio".
 - No conoce ni a "Command" ni a "Invoker“ */
class Receiver {
public:
    void action() { /* realiza la acción real */ }
};

/* ConcreteCommand:
 - Implementa la interfaz Command
 - Encapsula una solicitud concreta
 - Mantiene una referencia a Receiver
 - La acción queda encapsulada en el objeto comando.*/
class ConcreteCommand : public Command {
    Receiver* receiver;
public:
    ConcreteCommand(Receiver* r) : receiver(r) {}
    void execute() override { receiver->action(); }
};
```

Command – Implementación General (cont'd)

```
/* Invoker:
 - No sabe que hacen los comandos.
 - Almacena, organiza y ejecuta comandos.
 - Permite ejecutar multiples comandos en forma secuencial.
 - Sabe en qué orden y cuándo ejecutar los comandos.
*/
class Invoker {
    std::vector<std::unique_ptr<Command>> commands;
public:
    void addCommand(std::unique_ptr<Command> cmd) {
        commands.push_back(std::move(cmd));
    }

    void run() {
        for (auto& cmd : commands) cmd->execute();
    }
};
```

```
int main() {
    Receiver receiver;

    // Crear comando concreto
    auto command =
      std::make_unique<ConcreteCommand>(&receiver);

    // Invoker organiza la ejecución
    Invoker invoker;
    invoker.addCommand(std::move(command));

    // Ejecutar
    invoker.run();

    std::cout << "Comando ejecutado.\n";
    return 0;
}
```

Command – Ejemplo de Aplicación

```
#include <iostream>
#include <vector>
#include <memory>

// Command
class Command {
public:
   virtual void execute() = 0;
   virtual ~Command() {}
};

// Receiver
class Robot {
public:
   void moveForward() {
      std::cout << "Robot se mueve hacia adelante\n";
   }
   void turnLeft() {
      std::cout << "Robot gira a la izquierda\n";
   }
};
```

```
// Implementación de Commands
class MoveCommand : public Command {
   Robot* robot;
public:
   MoveCommand(Robot* r) : robot(r) {}
   void execute() override {
      robot->moveForward();
   }
};

class TurnCommand : public Command {
   Robot* robot;
public:
   TurnCommand(Robot* r) : robot(r) {}
   void execute() override {
      robot->turnLeft();
   }
};
```

Command – Ejemplo de Aplicación (cont'd)

```
// Invoker
class RemoteControl {
   std::vector<std::unique_ptr<Command>> commands;
public:
   void addCommand(std::unique_ptr<Command> cmd) { commands.push_back(std::move(cmd)); }
   void run() {
      for (auto& cmd : commands)
         cmd->execute();
   }
};

int main() {
   Robot r;
   RemoteControl remote;
   remote.addCommand(std::make_unique<MoveCommand>(&r));
   remote.addCommand(std::make_unique<TurnCommand>(&r));
   remote.addCommand(std::make_unique<MoveCommand>(&r));
   remote.run();
}
```

Resumen Patrones de Diseño de Comportamiento

Como se ha visto, los patrones de diseño de comportamiento tienen como objetivo desacoplar quién realiza una acción de cómo y cuándo se ejecuta, también presentan:

Alta Cohesión:

- Cada clase encapsula un comportamiento bien definido.
- Se separan responsabilidades de control, ejecución y reacción.

Bajo Acoplamiento:

- Los objetos interactúan mediante abstracciones y no referencias directas.
- Se reduce la dependencia entre emisor, receptor y lógica de decisión.

Evitan Código Duplicado:

- Los comportamientos se encapsulan y reutilizan.
- Se eliminan condicionales repetidos y lógica dispersa.

Bajo Impacto Ante Cambios:

- Nuevos comportamientos se agregan sin modificar clases existentes.
- Cambios en una política o acción no se propagan al sistema.

Preguntas?